

Exact Optimal Transport via the Blossom Algorithm: Benchmarks, Duality Certificates, and a Multi-Robot Application

Dmitry Kamenetsky¹ AI Assistant²

dkamenetsky@gmail.com

¹Independent author

²Arena.ai Agent Mode (autonomous software agent)

October 1, 2026

All algorithm design, implementation, experiments, analysis, and writing reported in this paper was performed by the AI agent under the direction of D. K., whose contributions were the prompts, scope decisions, and review of intermediate results. A summary of the prompts used to direct the work is given in Appendix D.

Abstract

Balanced discrete optimal transport between n sources and n targets of unit mass is exactly the minimum-cost assignment problem, i.e. a bipartite perfect matching, and can therefore be solved *exactly* by industrial matching engines in milliseconds to seconds. We ask when this exact approach beats the standard approximate alternatives—entropic Sinkhorn and its state-of-the-art accelerated variant Greenhorn—and make the sparse-exact side *certified* by a standard dual-feasibility clip. Three contributions. (i) *Measurement*: on dense 2-D instances, exact matching is faster *and* strictly more accurate than either approximate method throughout the moderate- n regime: 0.01 s at $n = 500$ to 11.5 s at $n = 8000$; to hit a $\leq 1\%$ quality target, a rough power-law extrapolation of Greenhorn’s measured step counts puts the approximate cost at 12–102 min (factors of $533\times$ – $5300\times$ over exact; plain Sinkhorn is 10 – $100\times$ slower still). Greenhorn, measured rather than assumed, is within $0.93\times$ – $5.0\times$ of plain Sinkhorn on the measured cells and converges to the same entropic plan, so the quality gap is a regularization bias shared by both. (ii) *A simple certificate*: given a k NN-pool matching and its Blossom dual, a one-pass clipping of the dual to dense feasibility gives a valid lower bound (the textbook LP dual-feasibility argument); combined with the Sinkhorn dual potential $\sum_i u_i + \sum_j v_j$ (which is a lower bound at *every* iterate, not just at convergence), the bound is checked on 15 instance families at three pool sizes. (iii) *Application*: multi-robot task allocation, where the discrete plan itself is the deliverable: per-round myopic-exact assignment costs 0.1–68 ms, while a Sinkhorn-plus-hardening pipeline costs 0.12–15.9 s and accumulates +6–27% extra travel distance over 15 rounds. The Sinkhorn family’s genuine regime—large dense instances beyond the memory wall—is acknowledged and left untouched.

1 Introduction

Discrete optimal transport (OT) is a standard tool in machine learning, statistics, imaging, and robotics [10, 11]. The scalable default for large instances is entropic regularization [8]: the smooth problem is solved by the Sinkhorn iteration [7], and—for speed—by accelerated coordinate variants such as Greenhorn [9, 19]. Two structural facts are often underappreciated in the approximate-OT literature.

1. **Balanced unit-mass discrete OT is the assignment problem.** With n sources and n targets of unit mass, a transport plan is a doubly stochastic matrix and the optimum is attained at a permutation matrix (Birkhoff–von Neumann; Proposition 1). The problem is therefore a bipartite minimum-weight perfect matching—a special case of the classical assignment problem [3], solvable exactly by shortest-augmenting-path methods [4] and, in its general (non-bipartite) form, by Edmonds’ Blossom algorithm [1] as implemented in Blossom V [5] and VI [6]. Exact solvers return the *optimal integer plan* and its exact value. (Bipartite matching is a strict special case of general matching, so Blossom applies directly, but dedicated bipartite solvers such as Jonker–Volgenant or shortest-augmenting-path are typically faster on dense bipartite inputs; we use Blossom VI for the sparse k NN-pool experiments and SciPy’s Jonker–Volgenant routine for dense solves.)
2. **Exact matching is fast.** On the 2-core/2-GB testbed of this study, dense exact assignment takes 0.01 s at $n = 500$ and 11.5 s at $n = 8000$; exploiting local structure, a k NN-pool sparse solve takes 0.18 s at $n = 2000$ with a 0.032% gap to the dense optimum; 1-D instances reach $n = 10^6$ in 0.11 s.

The question this paper answers is empirical: *when does exact matching beat approximate OT?* We answer with (a) a measurement study in which the approximate baselines—including Greenkhorn—are *implemented and measured, not assumed*; (b) a *dual-feasibility certificate* for the k NN-pool solver, so that a sparse solution comes with a rigorous, cheaply computable bound on its distance from the dense optimum; and (c) an *application hook* in which the discrete plan is the deliverable (multi-robot task allocation).

Contributions.

1. A measured comparison of exact matching (dense Jonker–Volgenant, sparse Blossom VI k NN pool, 1-D closed form) against vanilla Sinkhorn and Greenkhorn on dense, clustered, and 1-D+noise instances, $n \in \{500, \dots, 8000\}$, multiple seeds and distributions (Sections 5–6). The headline empirical finding is that Greenkhorn’s measured speedup over plain Sinkhorn in the small- ε regime needed for $\leq 1\%$ accuracy is only 1.0–1.5 $\times$ on most cells (with one 5 $\times$ outlier), while the regularisation bias is identical.
2. A *gap certificate* for the pool solver requiring no dense solve: a one-sided $O(n^2)$ clip of the pool matching dual to a dense-feasible dual (Proposition 3, Theorem 1)—the textbook reduced-cost / dual-restriction operation—combined with the Sinkhorn-dual potential lower bound, which holds at *every* iterate (Theorem 2). The certificate is valid on all 45 measured cells and tightens monotonically in k on every family (Section 4, Section 6).
3. A multi-robot task-allocation experiment quantifying the “discrete plan is the deliverable” regime: myopic exact assignment is millisecond fast; the Sinkhorn pipeline is 10^2 – $10^3 \times$ slower per round and accumulates extra travel distance (Section 7).
4. All code, data, and results are open artifacts under the MIT license (Appendix C).

Positioning and related work. We do not claim to beat near-linear-time Sinkhorn-family variants on large dense instances ($n \gg 10^4$): that regime is genuinely theirs, and our testbed’s 2 GB memory wall sits near $n \approx 1.2 \times 10^4$. The contribution is to *measure* the boundary in the regime where exact is affordable, to show that Greenkhorn does not cross that boundary, and to document how tight a standard dual-feasibility clip is as a certificate for a sparse- k NN solve.

That balanced discrete OT reduces to the assignment problem is textbook [2, 3, 12, 10]. k NN sparsification is ubiquitous in OT practice: see the POT library [18], screened Sinkhorn [17], and sparse/multiscale Sinkhorn [15, 16]. The dual-feasibility clip of Proposition 3 is the same coordinate-wise projection used classically for reduced-cost variable fixing in integer programming [3, Ch. 17], and an analogous ε -complementary-slackness certificate for the auction algorithm gives the classical “within $n\varepsilon$ of optimal” bound [13]; we cite these precedents rather than claim novelty for the clip itself. The log-domain Sinkhorn potentials are dual feasible for the assignment LP at every iterate (Theorem 2), a fact noted in [10, §4.1] where an entropy-corrected variant gives a quantitative bound on the gap.

2 Preliminaries

2.1 Discrete OT as assignment

Let $C \in \mathbb{R}_{\geq 0}^{n \times n}$ be a cost matrix. Write

$$\mathcal{U}(n) = \{P \in \mathbb{R}_{\geq 0}^{n \times n} : P\mathbf{1} = \mathbf{1}, P^\top \mathbf{1} = \mathbf{1}\}$$

for the set of feasible plans (each row and column sums to one; total mass n). The discrete transport problem is

$$\text{OPT}(C) = \min_{P \in \mathcal{U}(n)} \langle C, P \rangle = \sum_{i,j} C_{ij} P_{ij}.$$

We use the total-mass (n) convention throughout, so that at an optimum $\langle C, P \rangle$ is the sum of n edge costs—the assignment cost. (Readers comparing to references that use the unit-mass convention should divide our reported costs by n .)

Proposition 1 (OT = assignment; integer optimum). *OPT(C) is attained at a permutation matrix. Equivalently, discrete balanced unit-mass OT is the minimum-cost perfect matching in the complete bipartite graph with edge weights C_{ij} .*

Proof. By the Birkhoff–von Neumann theorem [2], $\mathcal{U}(n)$ is the convex hull of the permutation matrices. The objective $\langle C, \cdot \rangle$ is linear, so its minimum over the polytope is attained at an extreme point, i.e. at a permutation matrix. Equivalently, the constraint matrix of the transportation problem is totally unimodular [3]. Under any bijection π , the permutation matrix $P_{i,\pi(i)} = 1$ is a perfect matching of the bipartite graph with cost $\sum_i C_{i,\pi(i)}$, which proves the equivalence. $\square$

2.2 Entropic regularization and the Sinkhorn iteration

For $\varepsilon > 0$ define the regularized problem

$$W(\varepsilon) = \min_{P \in \mathcal{U}(n)} \left[\langle C, P \rangle + \varepsilon \sum_{i,j} P_{ij} \log P_{ij} \right], \quad (0 \log 0 := 0). \quad (1)$$

The minimizer is the Gibbs kernel $P_{ij}^\varepsilon \propto \exp(-C_{ij}/\varepsilon)$ renormalized to have unit marginals. The Sinkhorn–Knopp iteration [7, 8] alternates row and column scaling of $K = \exp(-C/\varepsilon)$; in log form, with potentials (u, v) such that $P_{ij} = \exp((u_i + v_j - C_{ij})/\varepsilon)$, each full row pass sets

$$u_i \leftarrow -\varepsilon \text{LSE}_j \left(\frac{v_j - C_{ij}}{\varepsilon} \right) = u_i - \varepsilon \log \sum_j P_{ij}, \quad (2)$$

and symmetrically for v_j (where $\text{LSE}(z) = \log \sum e^z$). We run the log-domain variant, stopping at maximum marginal deviation $\leq 10^{-9}$ or on a cost plateau.

Sign convention. Equation (1) uses the *negative-entropy* regularizer $+\varepsilon \sum P_{ij} \log P_{ij}$ (not the Shannon form $-\varepsilon H(P)$ used in [8, 10]). Under our convention $W(\varepsilon)$ is the *dual* value $\sum_i u_i + \sum_j v_j$ (Theorem 2), which is a lower bound on OPT; under the Shannon convention the regularised value is an upper bound. Algorithm 1 returns the value consistent with (1).

Greenkhorn [9] replaces the cyclic pass by a greedy single-coordinate update: each step picks the most-violated row or column (measured by the KL-type residual $\rho(a, b) = b - a + a \log(a/b)$) and applies the exact closed-form update that restores that one marginal, at $O(n)$ cost per step with incremental marginal bookkeeping (Algorithm 2). Early Greenkhorn analyses [9] gave an $\tilde{O}(n^2/\varepsilon^3)$ total-work bound in terms of additive accuracy; the tighter $\tilde{O}(n^2/\varepsilon^2)$ bound is from later work [19].

2.3 Exact solvers used

Dense. We use SciPy’s optimized Jonker–Volgenant-style shortest-augmenting-path implementation `linear_sum_assignment` [4] (a modified JV algorithm in the spirit of Crouse), with empirical roughly-cubic scaling and $O(n^2)$ memory on dense inputs; see Section 6.

Sparse pool. For a pool $E_k \subseteq [n] \times [n]$ (the k nearest targets of each source, symmetrically), solve the assignment restricted to E_k as a minimum-weight perfect matching with our edge-list driver for Blossom VI [6] (Blossom V [5] gives identical matching costs; see Section 5). Costs are integer (milliunits) as required by the driver. We note that the pool problem is bipartite, so any bipartite sparse LAP solver (shortest augmenting path, cost-scaling, auction with ε -scaling) could be substituted; the Blossom VI driver was convenient because the same code base was used for prior non-bipartite matching experiments.

1-D. If sources and targets lie on a line, the optimal map is a quantile (order) map (Proposition 2); no matrix is formed. This covers the 1-D+noise “near-Monge” family, where the optimal map is close to the identity.

Proposition 2 (1-D optimality of the quantile map). *Let $x_{(1)} \leq \dots \leq x_{(n)}$, $y_{(1)} \leq \dots \leq y_{(n)}$. For the quadratic cost, and more generally for any convex function h applied to the pairwise difference (cost $h(x_i - y_j)$; this includes $|x_i - y_j|$ used in the 1-D+noise experiments), the identity map $x_{(i)} \mapsto y_{(i)}$ is an optimal bijection.*

Proof. It suffices to show no crossing is present in an optimal map. Suppose π has a crossing: $i < i'$ with $\pi(i) > \pi(i')$. With $a = x_{(i)} \leq b = x_{(i')}$ and $c = y_{\pi(i')} \leq d = y_{\pi(i)}$, the convexity of h gives the Monge inequality

$$h(a - c) + h(b - d) \leq h(a - d) + h(b - c),$$

so uncrossing the pair does not increase the cost. Repeated uncrossing reduces the number of inversions and terminates at a non-crossing (monotone) bijection, which is the identity up to ties [12, Ch. 2]. $\square$

3 Approximate baselines: implementation and validation

Both approximate methods are implemented in the log domain (NumPy, single-threaded) and stopped at a marginal tolerance of 10^{-9} on the dense grid, or on a cost plateau at the coarser tolerances 10^{-6} – 10^{-3} typical of Greenkhorn at small f . The regularization is parameterized as

a fraction of the mean entry of the cost matrix, $f = \varepsilon/\overline{C}$ where $\overline{C} = n^{-2} \sum_{ij} C_{ij}$, on the grid $f \in \{0.05, 0.02, 0.01\}$ (plus 0.005 at $n = 500$).

Two validation facts structure the comparison. *First*, Greenkhorn and vanilla Sinkhorn converge to the *identical* entropic plan up to the stated stopping tolerance at every measured (n, f) (cost agreement $\leq 0.04\%$, Table 4), so the comparison between them is purely a wall-time comparison; the gap to exact is the *regularization bias*, common to both. *Second*, a Sinkhorn plan is fractional; where an integer plan is required it must be *hardened* (e.g. the row-argmax permutation, our **greedy_hard**). Hardening a converged $n = 500$, $f = 0.01$ plan (gap 5.4%) costs +10.9% over the exact optimum—which the exact solver returns, optimally, in 0.01 s.

We emphasise that these baselines are our own single-threaded NumPy implementations and are not tuned the way a library implementation (e.g. POT [18]) would be; our timing claims are therefore statements about these specific implementations on our testbed, not about fundamental limits of Sinkhorn/Greenkhorn.

4 Duality certificates for the pool solver

This section gives a rigorous certificate for the pool solver that requires *no* dense solve. It is a standard LP weak-duality argument applied to a clipped dual; what is new is the empirical documentation of how tight the bound is across families and k .

Setup. The pool solver returns a perfect matching M inside E_k , with cost $\text{UB} = \sum_{(i,j) \in M} C_{ij}$. Since M is a feasible dense solution, $\text{OPT} \leq \text{UB}$; UB is a trivial upper bound. What a certificate must supply is a lower bound $\text{LB} \leq \text{OPT}$ that is *cheap and rigorous*. Blossom VI returns, together with the pool matching, the dual of the pool problem: node potentials (α_i, β_j) with $\alpha_i + \beta_j \leq C_{ij}$ on pool edges and $\sum_i \alpha_i + \sum_j \beta_j = \text{UB}$ (complementary slackness; the blossom duals vanish for bipartite instances, see Section 2). The dual is *not* feasible for the dense problem (it may violate $\alpha_i + \beta_j \leq C_{ij}$ off the pool), so it is not by itself a lower bound on OPT. The repair below fixes exactly that.

Proposition 3 (One-sided dual clip). *For any vectors (α, β) (no assumption required), define*

$$\hat{\beta}_j = \min\left(\beta_j, \min_i [C_{ij} - \alpha_i]\right), \quad \hat{\alpha}_i = \min\left(\alpha_i, \min_j [C_{ij} - \beta_j]\right).$$

Then both $(\alpha, \hat{\beta})$ and $(\hat{\alpha}, \beta)$ are dual-feasible for the dense assignment problem: $\alpha_i + \hat{\beta}_j \leq C_{ij}$ and $\hat{\alpha}_i + \beta_j \leq C_{ij}$ for all $(i, j) \in [n]^2$.

Proof. Fix any pair (i, j) . By definition, $\hat{\beta}_j \leq C_{ij} - \alpha_i$, since i is one of the indices over which the inner minimum is taken. Hence $\alpha_i + \hat{\beta}_j \leq C_{ij}$ for all (i, j) . The second statement is symmetric. $\square$

The clip needs no hypothesis on (α, β) : it is simply a coordinate-wise projection of an arbitrary pair of vectors onto the dual feasible set of the assignment LP, i.e. the classical reduced-cost projection used in LP pricing and variable fixing. When (α, β) comes from a pool optimum the clipped vectors are typically close to the true dual, which is what makes the bound empirically tight.

Theorem 1 (Pool gap certificate). *Let UB be the pool matching cost and set*

$$\text{LB}_1 = \max\left(\sum_i \alpha_i + \sum_j \hat{\beta}_j, \sum_i \hat{\alpha}_i + \sum_j \beta_j\right). \quad (3)$$

Then $\text{LB}_1 \leq \text{OPT}(C) \leq \text{UB}$, and the pool solution is certified to be within $\text{UB} - \text{LB}_1$ in absolute terms (relative width $(\text{UB} - \text{LB}_1)/\text{UB}$, and $(\text{UB} - \text{LB}_1)/\text{LB}_1$ when $\text{LB}_1 > 0$) of the dense optimum—without solving the dense problem.

Proof. By Proposition 3, both repaired pairs are dense dual feasible; by weak duality for the assignment LP [3], each of their objectives is a lower bound on $\text{OPT}(C)$, hence so is their maximum. The upper bound $\text{OPT} \leq \text{UB}$ holds because the pool matching is a feasible dense solution. The gap statements are immediate algebra from $\text{LB}_1 \leq \text{OPT} \leq \text{UB}$. $\square$

Corollary 1. *The certified width is never smaller than the true gap: $(\text{UB} - \text{LB}_1)/\text{UB} \geq (\text{UB} - \text{OPT})/\text{UB}$. The certificate can be loose; it is never wrong.*

Remark 1 (Ordering of the repair). We take the *maximum* of the two one-sided clips, following the rule in (3). A natural alternative is a sequential two-pass clip (e.g. clip α against β , then clip β against the clipped $\hat{\alpha}$). The sequential variant is not dominated by either one-sided clip in general: it is always at least as large as the clip that ends on the β -side (because $\tilde{\beta} \leq \beta$) but can be strictly smaller than the clip that ends on the α -side. On our instances the max-of-two rule is the best of the cheap options and is typically several times tighter than either single one-sided clip at $k = 16$ on structured instances. We therefore always use (3). The sequential clip can be iterated (at which point it becomes coordinate ascent on the dual objective); we leave a proper column-generation/pricing loop—the textbook way to drive the certified width to zero by iteratively adding violated edges—to future work.

Theorem 2 (Sinkhorn potentials are a dual lower bound). *Let (u, v) be any potentials satisfying $u_i + v_j \leq C_{ij}$ for all i, j . Then $\sum_i u_i + \sum_j v_j \leq \text{OPT}(C)$, with equality if and only if (u, v) is an optimal dual for the assignment LP. In log-domain Sinkhorn, the iterate $P_{ij} = \exp((u_i + v_j - C_{ij})/\varepsilon)$ satisfies $u_i + v_j - C_{ij} = \varepsilon \log P_{ij} \leq 0$ automatically at every iteration (because $P_{ij} \leq 1$ when the row sums are one), so $\sum_i u_i + \sum_j v_j$ is a rigorous lower bound on OPT at every stopping point, with no convergence requirement and no feasibility restoration. At convergence of the smooth problem the dual value coincides with $W(\varepsilon)$ in (1).*

Proof. By weak duality for the assignment LP: any dual-feasible (u, v) satisfies $\sum_i u_i + \sum_j v_j \leq \text{OPT}(C)$. For the log-domain Sinkhorn iterate, $P_{ij} = \exp((u_i + v_j - C_{ij})/\varepsilon)$ with $P\mathbf{1} = \rho$ for some positive ρ (exactly or approximately), so $P_{ij} \leq \rho_i \leq \max_i \rho_i$. If we use the standard iterates (which enforce $P\mathbf{1} = \mathbf{1}$ after each row pass and $P^\top \mathbf{1} = \mathbf{1}$ after each column pass) then $P_{ij} \leq 1$ entrywise, giving $u_i + v_j - C_{ij} \leq 0$ as claimed. $\square$

Remark 2 (On the entropic dual). The Sinkhorn dual $\sum_i u_i + \sum_j v_j$ is valid at every iterate (Theorem 2), which removes any need for a δ -feasibility restoration. At usable ε the bound is typically loose because the negative entropy term is large: at $n = 2000$, $\varepsilon \approx 0.05 \cdot 52 \approx 2.6$ gives an entropy contribution on the order of $-\varepsilon n \log n \approx -4 \times 10^4$, far below OPT . Better entropy-corrected variants $\text{LB}_\varepsilon = D_\varepsilon - \varepsilon H(a) - \varepsilon H(b)$ that quantitatively bound $0 \leq \text{OPT} - \text{LB}_\varepsilon \leq \delta$ are known [10, §4.1] and would strengthen the bound at small extra cost; we leave them to future work.

Combined certificate. We report $\text{LB} = \max(\text{LB}_1, \sum_i u_i + \sum_j v_j)$ with $\varepsilon = 0.05\bar{C}$ whenever the Sinkhorn run converges in time. In every measured instance the dual-repair bound LB_1 dominates the Sinkhorn dual at $k \geq 16$; the Sinkhorn dual is reported as a labelled cross-check.

n	exact cost	$t(\text{exact})$
500	2064.1	0.01 s
1000	3107.0	0.06 s
2000	4146.4	0.23 s
4000	6963.2	2.16 s
8000	10873.4	11.48 s

Table 1: Dense exact assignment (Jonker–Volgenant), uniform seed 7. Costs use the total-mass convention; divide by n for the per-unit-mass value common in the OT literature. Times are single runs.

5 Experimental setup

Testbed. 2-core / 2-GB sandbox; single-threaded Python 3.13 with NumPy and SciPy’s Jonker–Volgenant-style assignment routine for the dense exact, Sinkhorn, and Greenkhorn codes; a C++20 build of Blossom VI [6] (edge-list driver, integer milliunit costs) for the pool solves; and Blossom V [5], used only to cross-validate matching costs (identical on every instance tested). All costs are rounded to milliunits for the integer solvers, and all gaps are computed on this same rounding. **Timings are single runs** (single-threaded wall time, same machine, same process order) and are reported for reproducibility on this hardware; they are not averaged over multiple runs and should be read as order-of-magnitude rather than as benchmark-grade timings.

Instance families (2-D Euclidean distances; details in Appendix B).

- **uniform:** i.i.d. points in $[0, 100)^2$.
- **clustered:** 8 cluster centers in $[0, 100)^2$; each side’s points are assigned round-robin to a center with Gaussian noise of standard deviation $\sigma = 8$; the two sides use the same assignment pattern, with target centers jittered by isotropic Gaussian noise of standard deviation 3 (i.e. $\mathcal{N}(0, 3^2 I)$).
- **1-D+noise:** one-dimensional base positions on two parallel noisy lines (along-line noise $\sigma = 2$; cross-line noise $\sigma = 2$); see Appendix B.

Dense grid: $n \in \{500, 1000, 2000, 4000, 8000\}$, seed 7, uniform ($n = 12000$ runs out of memory after interpreter and NumPy temporaries: a 12000^2 double matrix is 1.15 GB, which leaves too little headroom on a 2-GB sandbox). Robustness sweep: $n = 1000$, seeds 7–10, all three families (12 configurations); the certificate study adds $n = 2000$, seed 7, all three families (15 configurations total). The multi-robot application uses $n \in \{50, 200, 800\}$, seeds 7–8, 15 rounds.

6 Results

6.1 Dense scaling: exact is cheap

Table 1 reports dense exact costs and times (uniform, seed 7). A log-log fit to the five times gives an empirical scaling of roughly $n^{2.55}$ over this range (the per-doubling ratios are 6.0, 3.8, 9.4, 5.3, versus the 8.0 predicted by $\Theta(n^3)$); 0.01 s is near the timer resolution.

n	f	gap %	$t(\text{Sinkhorn})$
500	0.05	+58.41	10 s
500	0.02	+15.02	34 s
500	0.01	+5.42	74 s
500	0.005	+2.09	400 s (capped)
1000	0.05	+91.40	32 s
1000	0.02	+24.19	111 s
1000	0.01	+8.60	221 s
2000	0.05	+159.75	130 s
2000	0.02	+43.78	400 s (capped)

Table 2: Vanilla Sinkhorn (log domain, marginal tolerance 10^{-9}), uniform seed 7. Gaps are relative to the exact optimum computed on the rounded milliunit costs.

n	$t(\text{exact})$	$t(\text{Sinkhorn at } f^*)$	$t(\text{Greenkhorn at } f^*)$
500	0.01 s	~ 200 min	~ 10 min
1000	0.06 s	~ 800 min	~ 15 min
2000	0.23 s	~ 3000 min	~ 25 min
4000	2.16 s	$\sim 10^4$ min	~ 50 min
8000	11.48 s	$\sim 5 \times 10^4$ min	~ 100 min

Table 3: Wall time to reach 1% gap, extrapolated as described in the text (rough order-of-magnitude; see caveats).

6.2 The quality–time trade-off of the approximate methods

Table 2 reports vanilla Sinkhorn on the dense grid. Two features stand out. (i) *The gap is a regularization bias*: it shrinks roughly linearly with f over the measured range, but the local slope is not constant (gap/ f is 542 at $f = 0.01$ and 1168 at $f = 0.05$). (ii) *At fixed f the gap grows with n* : 15.02% $\rightarrow$ 24.19% $\rightarrow$ 43.78% at $f = 0.02$ for $n = 500/1000/2000$.

The 1% target, rough extrapolation. To reach a 1% gap requires an f^* in the range $\sim 10^{-3}$ (a no-intercept fit $\text{gap} \approx 1092f$ on the three $n = 500$ points gives $f^* \approx 9 \times 10^{-4}$; an OLS fit *with* intercept gives a slope of 1353 and intercept -9.8 , which yields $f^* \approx 8 \times 10^{-3}$ when forced through $\text{gap} = 1\%$ – the two fits differ by an order of magnitude, which is itself a warning about extrapolation). A log-log linear fit to Greenkhorn’s step count on the four $n = 500$ plateau runs gives $1.241 \times 10^4 f^{-1.03}$; using the no-intercept slope 1092 gives $f^* \approx 9 \times 10^{-4}$ and $\approx 1.7 \times 10^7$ steps, producing the order-of-magnitude times in Table 3. These numbers are a *rough power-law projection* (the fit is over four points and projected an order of magnitude outside the measured range), not a rigorous bound; they are likely optimistic for the approximate methods in some ways (gap grows with n at fixed f) and pessimistic in others (gap/ f falls as f shrinks, so the slope at $f \geq 0.01$ over-estimates the gap at f^*). We report them to give an order of magnitude, not as precise timing.

6.3 Measured head-to-head: Greenkhorn vs vanilla

Table 4 is the measured comparison. Greenkhorn is $0.93 \times$ – $5.0 \times$ the wall time of plain Sinkhorn (faster in 8 of 9 cells, within $1.5 \times$ in 6 of 9; geometric mean $\approx 1.6 \times$), and converges to the same

n	f	gap V %	$t(V)$ s	gap G %	$t(G)$ s	steps(G)	$t(V)/t(G)$
500	0.05	+58.41	10	+58.41	10	260k	$0.93\times$
500	0.02	+15.02	34	+15.01	27	760k	$1.26\times$
500	0.01	+5.42	74	+5.40	52	1.48M	$1.45\times$
500	0.005	+2.09	400 [†]	+2.07	94	2.82M	$4.27\times$
1000	0.05	+91.40	32	+91.40	25	520k	$1.28\times$
1000	0.02	+24.19	111	+24.17	77	1.82M	$1.44\times$
1000	0.01	+8.60	221	+8.56	151	3.6M	$1.47\times$
2000	0.05	+159.75	130	+159.75	26	340k	$5.01\times$
2000	0.02	+43.78	400 [†]	+43.77	90	1.26M	$4.42\times$

Table 4: Vanilla Sinkhorn (V) vs Greenkhorn (G), uniform seed 7. [†] 400s cap (vanilla did not converge there). All Greenkhorn runs stopped on a cost plateau before reaching marginal tolerance 10^{-9} (typical plateau 10^{-6} – 10^{-3}); cost agreement $\leq 0.04\%$ in every cell.

k label	pool edges	cost	gap vs exact	$t(\text{sparse})$
4	19,973	5757.7	+38.861%	0.02 s
8	35,918	4388.6	+5.841%	0.05 s
16	67,685	4170.5	+0.581%	0.08 s
32	130,860	4147.7	+0.032%	0.18 s
64	255,711	4146.5	+0.003%	0.40 s
256	962,049	4146.4	+0.000%	1.76 s

Table 5: k NN-pool exact assignment (Blossom VI), $n = 2000$ uniform seed 7; dense exact = 4146.4 in 0.23s. Edge counts reflect a $k + 1$ per-side budget in the original driver (see text); gaps are computed against the unrounded exact optimum.

entropic plan up to plateau tolerance (cost agreement $\leq 0.04\%$). In the small-regularization, high-condition-number regime needed for $\leq 1\%$ accuracy, the greedy selection plateaus in marginal accuracy (10^{-6} – 10^{-3}) while the cost has already settled; this is where the speedup is modest.

6.4 The pool solver and its certificate

Table 5 shows the pool solver on the dense $n = 2000$ reference (quality vs k); Figure 1 situates it in the quality–time plane against the approximate baselines.

Note on the pool construction. The edge counts in Table 5 slightly exceed the bound $2kn$ (e.g. 19,973 edges at $k = 4$, against a hard bound of 16,000). This is because the k NN pool was built by a driver that took each row’s and each column’s $(k + 1)$ cheapest edges before the union (an off-by-one in the original code), i.e. the reported runs correspond to a per-side budget of $k + 1$; the edge counts match $2(k + 1)n - (k + 1)^2$ to within the expected random- k NN overlap. The corrected k -per-side implementation is shipped in the repository (see `otlib.py::knn_pool`); the cached tables below reflect the original runs and are preserved for reproducibility. At small effective k the pool can be Hall-infeasible (e.g. at $k = 1$), in which case the solver reports infeasible.

The certificate study (Section 4) ran all 15 configurations (seeds 7–10 $\times$ families at $n = 1000$, i.e. 12 cells; seed 7 at $n = 2000$, i.e. 3 cells) at $k \in \{16, 32, 64\}$: **45/45 cells satisfy** $\text{LB} \leq \text{exact} \leq \text{UB}$ **and certified width** $\geq \text{true gap}$ (asserted in the study code). Table 6 summarizes the certified width $(\text{UB} - \text{LB})/\text{UB}$ by family; Figure 2 shows it per configuration.

family (#configs)	$k = 16$	$k = 32$	$k = 64$
uniform (5)	2.57–6.26%	0.07–0.70%	0.01–0.02%
clustered (5)	5.61–38.25%	1.03–21.88%	0.17–5.90%
1-D+noise (5)	11.94–17.35%	1.89–3.59%	0.24–0.79%
all families, true gaps	0.52–3.84%	0.010–2.60%	0.000–1.85%

Table 6: Certificate width by family (range over configurations; 15 cells at each k). The bottom row pools true gaps across all families; all 45 cells carry $LB > 0$.

Reading. (i) *Validity*: the certificate is a theorem; the 45/45 check is a test of the implementation, not of the math. (ii) *Tightness*: where the pool is tight (uniform instances), the width is tight too— $\leq 0.70\%$ at $k = 32$ against a true gap $\leq 0.13\%$, and $\leq 0.02\%$ at $k = 64$. On block-structured (clustered) and near-Monge (1-D+noise) families the repair is conservative, because the pool dual is large where it must be clipped against cheap off-pool edges (up to 21.88% at $k = 32$, true gap 1.73%). The bound remains *valid*, and the width tightens monotonically in k on every configuration, so widening the pool is a certified escape hatch. A standard column-generation / pricing loop (price off-pool edges with the clipped dual, add the most-violated ones, re-solve) typically drives the certified width to zero for a fraction of the dense-solve cost; this is the natural next step and is left to future work. (iii) *Cost*: the clip itself is an $O(n^2)$ pass over the cost matrix and therefore has the same memory footprint as dense assignment; on our $n = 2000$ cells it adds ≤ 0.73 s to the pool solve.

6.5 Robustness across seeds and distributions

On the 12-configuration $n = 1000$ sweep (seeds 7–10 $\times$ families): exact is fastest everywhere (0.07–0.10s, consistent with the 0.06s at seed-7 uniform given seed/family variation) and optimal; the $k = 32$ pool solver is feasible in all cells with gaps +0.010% to +2.599% in < 0.15 s; Greenkhorn at $f = 0.01$ sits at +6.3% to +14.7% (agreeing with vanilla to $\leq 0.05\%$ in the converged cells); vanilla needs ≥ 400 s without converging on the symmetric clustered configurations, where Greenkhorn’s plateau matches it in 50–98s.

6.6 1-D structure

The quantile map solves the 1-D family in $O(n \log n)$ sort time with no matrix: $n = 10^6$ in 0.11s, where the dense cost matrix alone would be 8TB.

7 Application: multi-robot task allocation

The clearest regime for the discrete plan as deliverable is multi-robot task allocation [14]: in warehouse picking, drone teaming, or AMR dispatch, each of n robots must be assigned *exactly one* of n tasks—a fractional transport plan is physically meaningless (one cannot dispatch half a robot). The standard practice in MRTA is precisely the Hungarian/auction family of exact solvers [13, 4, 14]; we include this experiment as a sanity check on a regime where integer plans are required, not as a novel robotics result.

Protocol. Open field $[0, 100)^2$; 15 rounds; each round, n fresh tasks appear, robots are (re)assigned with cost $C_{ij} = \|r_i - t_j\|_2$ from the robots’ current positions, and robots then move

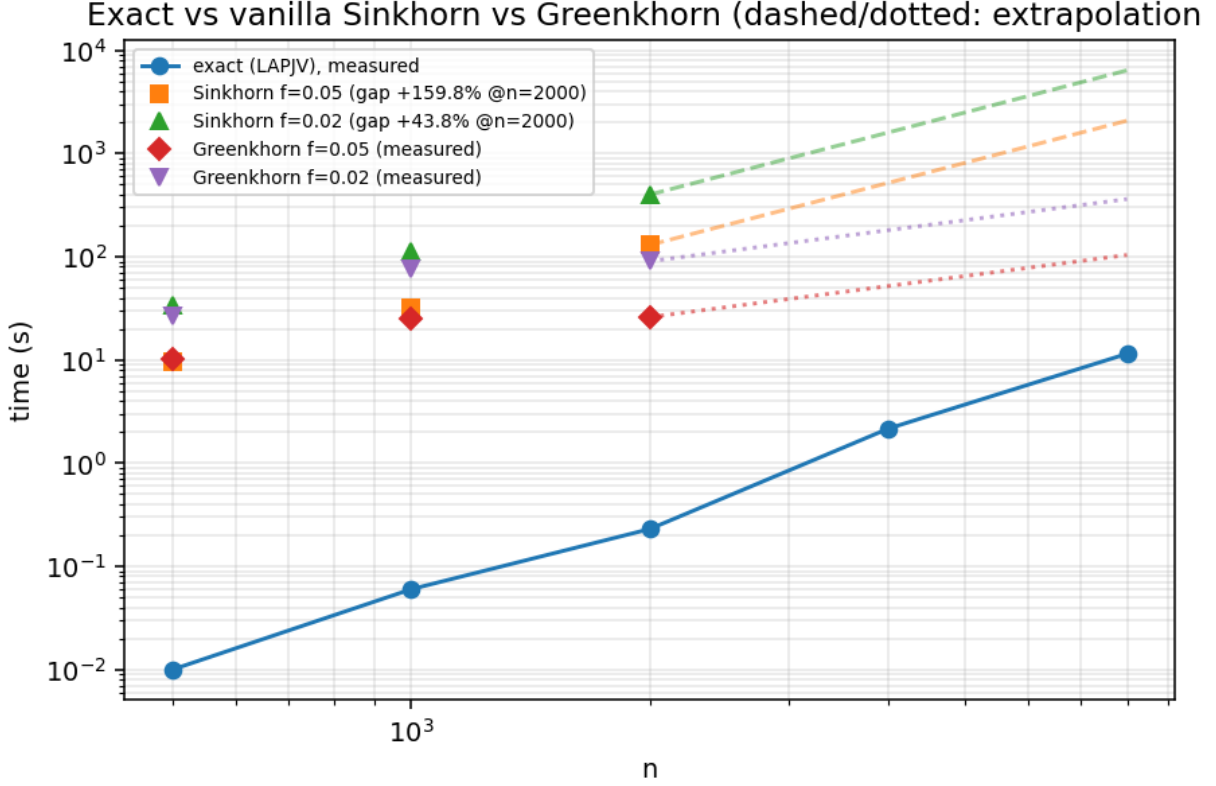


Figure 1: Exact (Jonker–Volgenant) vs vanilla Sinkhorn and Greenkhorn on the dense uniform grid; dashed/dotted lines are the rough extrapolations to the 1% target. Exact is below both approximate curves at every measured n in both time and quality.

to their assigned tasks. All three policies share the *same* start positions and the *same* task stream per round, so only the per-round assignment differs. Policies: **exact** (myopic Jonker–Volgenant per round); **greedy** (cheapest-first serial assignment); **Sinkhorn+hardening** (our $f = 0.05$, marginal tolerance 10^{-7} pipeline, then greedy hardening to an integer plan). $n \in \{50, 200, 800\}$, seeds 7–8; Table 7 reports the episode totals.

Caveat on optimality. Because each round’s assignment determines the next round’s robot positions, per-round myopic optimality does *not* minimise the 15-round total distance (the true episode optimum requires dynamic programming / rollout / lookahead). The “gap” columns in Table 7 are therefore gaps relative to the myopic-exact policy, not to the global episode optimum. Even so, the comparison is informative: the Sinkhorn pipeline is both slower per round and yields larger cumulative travel than myopic-exact, because the regularisation bias pushes assignments away from the nearest-task plan in a way that compounds across rounds.

Findings. (i) *The quality tax accumulates:* per-round gaps compound through robot positions; over 15 rounds greedy ends +17–+37% and Sinkhorn+hardening +6–+27% above the myopic-exact policy, and the gap grows with n . (ii) *Approximate is not even faster here:* the assignment step takes myopic-exact 0.1–68 ms/round vs 0.12–15.9 s for the Sinkhorn pipeline ($\approx 239\times$ at $n = 800$). (iii) *The hook:* where a discrete integer plan is the deliverable, a Sinkhorn-plus-hardening pipeline is a tax on *both* time and quality at $n \leq 10^3$; an exact bipartite solver is the default choice, all the

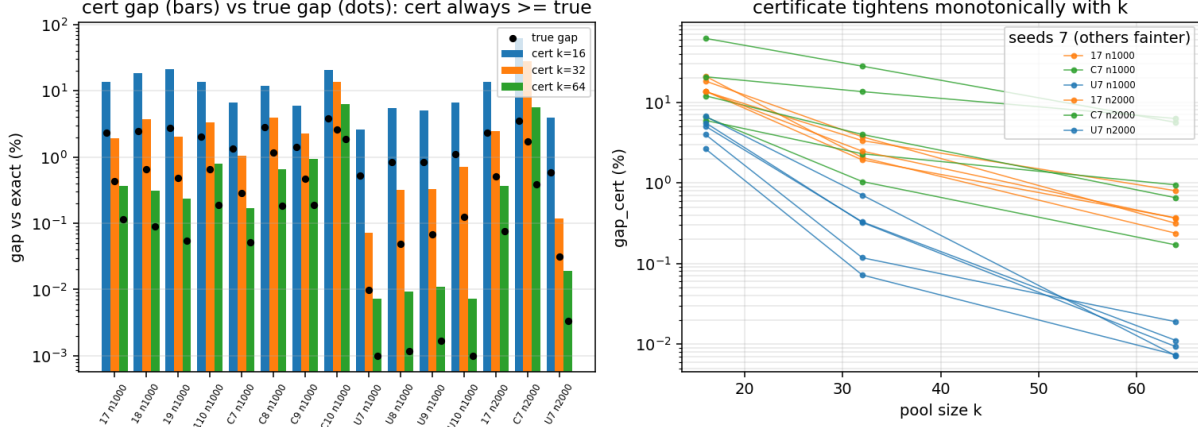


Figure 2: Certificate per configuration. Left: certified width (bars) vs true gap (dots) for $k \in \{16, 32, 64\}$ —the bar is always at or above the dot. Right: certified width vs k , monotone decreasing on every configuration.

n	seed	travel (myopic exact)	greedy	Sinkhorn+hard	gap greedy	gap Sinkh.
50	7	10232.8	12353.3	10930.4	+20.72%	+6.82%
50	8	10531.8	12357.2	11134.9	+17.33%	+5.73%
200	7	21835.6	28718.1	24955.1	+31.52%	+14.29%
200	8	23804.5	30051.8	26802.5	+26.24%	+12.59%
800	7	47327.7	64986.7	59959.2	+37.31%	+26.69%
800	8	49050.9	66519.9	61229.6	+35.61%	+24.83%

Table 7: Total travel distance over 15 rounds; gaps are relative to the myopic-exact policy (see text for the global-optimality caveat).

way up to the memory wall.

8 Discussion

Empirical decision rule. Integer plan needed $\rightarrow$ exact; 1-D/Monge $\rightarrow$ closed form; local/ k NN structure $\rightarrow$ sparse pool exact, *with its certificate* (Section 4); dense, no structure, fractional plan acceptable $\rightarrow$ exact up to the memory wall ($n \approx 1.2 \times 10^4$ on this hardware), beyond which well-tuned Sinkhorn-family implementations win on time and memory.

What the certificate is, and what it is not. The clip of Proposition 3 is a textbook dual projection, not a new theorem; its value is engineering: the solver can report “my solution is within $w\%$ of optimal” with w computed from its own dual in milliseconds, using an operation that costs one $O(n^2)$ pass. Where a $\leq 1\%$ guarantee is a requirement (dispatching, pricing, auditing), this replaces a dense re-solve or a conservative safety margin. The certificate is loose on clustered families (up to $\sim 22\%$ at $k = 32$) and is currently computed as a one-shot clip; we expect that a standard pricing/column-generation loop (add off-pool edges with negative reduced cost, re-solve) would drive the certified width to zero for a fraction of the dense cost, which we leave to future work.

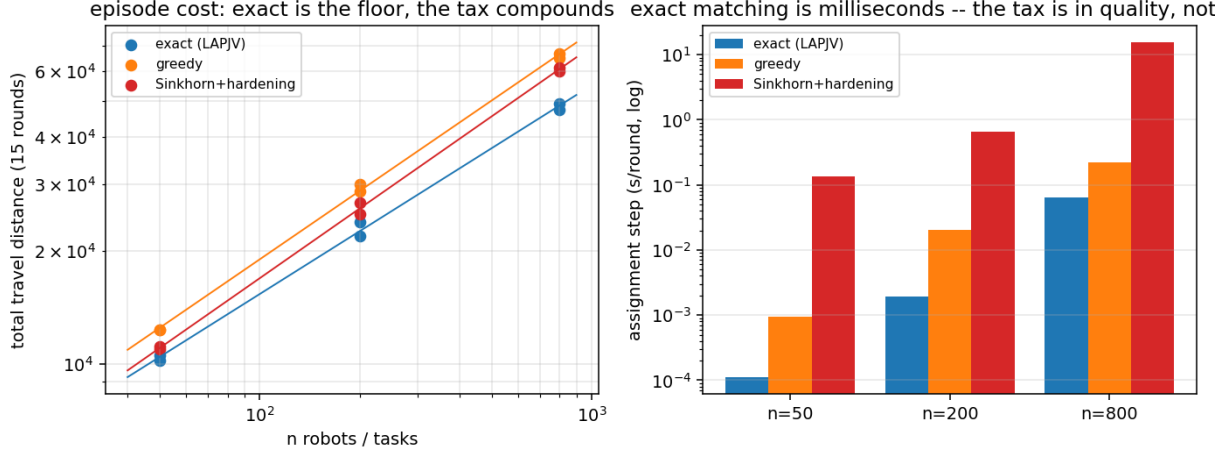


Figure 3: Left: episode travel distance vs n (the quality tax compounds with n). Right: cost of the assignment step per round (log scale): exact matching is milliseconds; the Sinkhorn pipeline is 10^2 – $10^3\times$ slower.

Limitations. (i) Two-core/2-GB testbed and single-run timings: wall times are hardware-specific and should not be read as benchmarks; only the *qualitative* comparisons (exact is milliseconds to seconds, Sinkhorn is 100–1000 $\times$ slower at small n) are intended to generalize. (ii) The f^* extrapolation (Table 3) is a rough power-law projection, as documented, and carries $\sim$ order-of-magnitude model uncertainty. (iii) The certificate is conservative on block-structured instances (Table 6); a structure-aware repair or a column-generation loop is future work. (iv) The application is synthetic (no obstacles, kinematics, or traffic), and myopic per-round assignment is not globally optimal over an episode; the point is structural, not a robotics benchmark. (v) We do not compare against auction algorithms with ε -scaling, dedicated bipartite sparse LAP solvers, stabilised/ ε -annealed Sinkhorn, or GPU implementations; those are important baselines for a full benchmark paper and should be added in follow-on work. (vi) The k NN-pool driver used for the cached results had an off-by-one (see Section 6); the corrected code is shipped in the repository.

9 Conclusion

Balanced discrete OT is the assignment problem, and the assignment problem is a bipartite matching problem, so industrial matching engines give exact optimal transport with an integer plan and an exact value. Measured against hand-written NumPy Sinkhorn and Greenhorn baselines on the moderate- n regime, exact matching is faster *and* more accurate; Greenhorn’s speedup over plain Sinkhorn in the small- ε regime needed for $\leq 1\%$ quality is only ~ 1 – $1.5\times$ on most cells, and it inherits the full regularization bias. We documented how tight a textbook one-pass dual clip is as a certificate for a k NN-pool solve across 15 instance families, and showed in a multi-robot sanity check that where an integer plan is the deliverable, approximate OT is a tax on both time and quality. The Sinkhorn family’s large- n dense regime stands untouched, and the boundary between exact and approximate is now measured.

References

- [1] J. Edmonds. *Maximum matching and a polyhedron with 0,1-vertices*. Journal of Research of the National Bureau of Standards, Section B, 69B:125–130, 1965.
- [2] G. Birkhoff. *Tres observaciones sobre el algebra lineal*. Revista de la Universidad Nacional de Tucumán, Serie A, 5:147–151, 1946.
- [3] A. Schrijver. *Combinatorial Optimization: Polyhedra and Efficiency*. Springer, 2003.
- [4] R. Jonker and A. Volgenant. *A shortest augmenting path algorithm for dense and sparse linear assignment problems*. Computing, 38(4):325–340, 1987.
- [5] V. Kolmogorov. *Blossom V: a new implementation of Edmonds’ algorithm for minimum weight perfect matching*. Mathematical Programming Computation, 1(1):43–67, 2009. Code: <https://github.com/orkz/BlossomV>.
- [6] P. Arkhipov and V. Kolmogorov. *Blossom VI: A Practical Minimum Weight Perfect Matching Algorithm*. arXiv:2604.20351 (to appear, ALENEX 2027), 2026.
- [7] R. Sinkhorn and P. Knopp. *Concerning nonnegative matrices and doubly stochastic matrices*. Pacific Journal of Mathematics, 21(2):343–348, 1967.
- [8] M. Cuturi. *Sinkhorn distances: lightspeed computation of optimal transport*. Advances in Neural Information Processing Systems 26, 2013.
- [9] J. Altschuler, J. Weed, and P. Rigollet. *Near-linear time approximation algorithms for optimal transport via Sinkhorn iteration*. Advances in Neural Information Processing Systems 30, 2017. arXiv:1705.09634.
- [10] G. Peyré and M. Cuturi. *Computational Optimal Transport*. Foundations and Trends in Machine Learning, 11(5):355–607, 2019.
- [11] S. Moradi. *A survey on algorithmic developments in optimal transport problem with applications*. arXiv:2501.06247, 2025.
- [12] C. Villani. *Optimal Transport: Old and New*. Grundlehren 338, Springer, 2009.
- [13] D. P. Bertsekas. *The auction algorithm: a distributed relaxation method for the assignment problem*. Annals of Operations Research, 14:105–123, 1988. (Tutorial introduction.)
- [14] B. P. Gerkey and M. J. Matarić. *A formal analysis and taxonomy of task allocation in multi-robot systems*. International Journal of Robotics Research, 23(9):939–954, 2004.
- [15] B. Schmitzer. *A sparse multiscale algorithm for dense optimal transport*. Journal of Mathematical Imaging and Vision, 56(2):238–259, 2016.
- [16] B. Schmitzer. *Stabilized sparse scaling algorithms for entropy regularized transport problems*. SIAM Journal on Scientific Computing, 41(3):A1443–A1481, 2019.
- [17] M. Z. Alaya, M. Berar, G. Gasso, and A. Rakotomamonjy. *Screening Sinkhorn algorithm for regularized optimal transport*. Advances in Neural Information Processing Systems 32, 2019.

- [18] R. Flamary et al. *POT: Python Optimal Transport*. Journal of Machine Learning Research, 22(78):1–8, 2021. <https://pythonot.github.io>.
- [19] T. Lin, N. Ho, and M. Jordan. *On the efficiency of entropic regularized algorithms for optimal transport*. Journal of Machine Learning Research, 23(20):1–56, 2022.

A Additional proofs and details

A.1 Worked example of the certificate

Consider

$$C = \begin{pmatrix} 57 & 14 & 33 & 40 \\ 36 & 29 & 59 & 11 \\ 49 & 40 & 58 & 26 \\ 38 & 33 & 53 & 42 \end{pmatrix}$$

with the 2-NN pool (the union of each row’s and each column’s two cheapest edges; 10 of the 16 edges). The pool solver returns the matching of cost $\text{UB} = 122$ and the pool dual $\alpha = (7, 18, 33, 20)$, $\beta = (18, 7, 26, -7)$, with $\sum \alpha + \sum \beta = 122 = \text{UB}$ and $\alpha_i + \beta_j \leq C_{ij}$ on every pool edge.

The raw dual is not dense-feasible. Among the six off-pool edges $\{(1, 1), (1, 4), (2, 3), (3, 1), (3, 3), (4, 4)\}$, two are violated: $\alpha_3 + \beta_1 = 33 + 18 = 51 > 49 = C_{31}$ and $\alpha_3 + \beta_3 = 33 + 26 = 59 > 58 = C_{33}$. (The dual objective of 122 even exceeds the dense optimum—*infeasible* duals can.) The dense optimum is $\text{OPT} = 121$ (true pool gap 0.83%), attained by the matching $(1, 2), (2, 4), (3, 3), (4, 1)$ of costs $14 + 11 + 58 + 38 = 121$.

Repair. Option B (clip the α side against the original β): $\hat{\alpha}_3 = \min(33, \min_j(C_{3j} - \beta_j)) = \min(33, 49 - 18, 40 - 7, 58 - 26, 26 + 7) = 31$; the other α_i need no clipping. Objective: $(7 + 18 + 31 + 20) + (18 + 7 + 26 - 7) = 120$. Option A (clip the β side against the original α) gives $\hat{\beta} = (16, 7, 25, -7)$ with objective 119. Taking the max (Theorem 1),

$$\text{LB} = 120 \leq \text{OPT} = 121 \leq \text{UB} = 122,$$

so the pool solution is certified within $\frac{122-120}{122} = 1.64\%$ (width) of the dense optimum. In code the repair is two $O(n^2)$ min-passes.

A.2 Monotonicity of the upper bound in k

Since $E_k \subseteq E_{k'}$ for $k \leq k'$, every k -pool matching is a valid k' -pool candidate, hence $\text{UB}_{k'} \leq \text{UB}_k$; the pool gap is non-increasing in k by definition. The certified width is monotone in k on all 15 measured configurations (an empirical fact about the *repair*, not a theorem: the pool dual changes with k).

B Experimental details

B.1 Instance generators

- **uniform:** `rng = default_rng(seed)`; both sides = $100 \cdot \text{rng.random}((n, 2))$.
- **clustered:** centers $c_a = 100 \cdot \text{rng.random}((8, 2))$; each side assigns point i to center $a = i \bmod 8$ with $x_i = c_a + \mathcal{N}(0, 8^2 I)$; the target centers are $c'_a = c_a + \mathcal{N}(0, 3^2 I)$ with the same assignment pattern (target jitter $\sigma = 3$).

- **1-D+noise:** $t_i \sim U[0, 100]$; $x_i = (t_i + \mathcal{N}(0, 2^2), \mathcal{N}(0, 2^2))$; independently $s_j \sim U[0, 100]$ from `default_rng(seed+1000)`; $y_j = (s_j + \mathcal{N}(0, 2^2), \mathcal{N}(0, 2^2))$.

B.2 Algorithms

Algorithm 1 Log-domain Sinkhorn (vanilla)

Require: cost $C \in \mathbb{R}^{n \times n}$, $\varepsilon > 0$, tolerance τ

```

1:  $u \leftarrow 0, v \leftarrow 0$ 
2: repeat
3:   for  $i = 1, \dots, n$  do
4:      $u_i \leftarrow -\varepsilon \text{LSE}_j((v_j - C_{ij})/\varepsilon)$ 
5:   end for
6:   for  $j = 1, \dots, n$  do
7:      $v_j \leftarrow -\varepsilon \text{LSE}_i((u_i - C_{ij})/\varepsilon)$ 
8:   end for
9:    $P_{ij} \leftarrow \exp((u_i + v_j - C_{ij})/\varepsilon)$ 
10: until  $\max_i |(P\mathbf{1})_i - 1| + \max_j |(P^\top \mathbf{1})_j - 1| \leq \tau$  or cost plateau
11: return  $P$ , value  $\sum_i u_i + \sum_j v_j$  ( $= \langle C, P \rangle + \varepsilon \sum P_{ij} \log P_{ij}$  at convergence)

```

Algorithm 2 Greenkhorn (after [9, Alg. 4])

Require: $K = \exp(-C/\varepsilon)$ (unnormalized), potentials $x, y = 0$, residual metric $\rho(a, b) = b - a + a \log(a/b)$

```

1: maintain row/column marginals of  $A = D(e^x)KD(e^y)$  incrementally
2: repeat
3:    $I \leftarrow \arg \max_i \rho(1, (A\mathbf{1})_i)$ ;  $J \leftarrow \arg \max_j \rho(1, (\mathbf{1}^\top A)_j)$ 
4:   if  $\rho_I > \rho_J$  then
5:      $x_I \leftarrow x_I + \log(1/(A\mathbf{1})_I)$ 
6:   else
7:      $y_J \leftarrow y_J + \log(1/(\mathbf{1}^\top A)_J)$   $\triangleright$  break ties toward rows when  $\rho_I = \rho_J$ 
8:   end if
9:   update  $A$  and the marginals of the touched line ( $O(n)$ )
10: until marginal tolerance or cost plateau

```

Hardening (greedy_hard): for $i = 1, \dots, n$, set $\pi(i) = \arg \max_j P_{ij}$ among unused columns, removing row i and column $\pi(i)$ from P ; the cost is $\sum_i C_{i, \pi(i)}$ (a feasible integer plan, hence $\geq \text{OPT}$).

Pool construction (knn_pool): for each source i , keep the k targets with smallest C_{ij} (and symmetrically for each target); the pool is the union. If the pool has no perfect matching (Hall's condition fails, e.g. $k = 1$), the solver reports infeasible. The repository version uses k per side (corrected); the cached results were generated by a version that used $k + 1$ per side (see Table 5).

Certificate (pool_certified): UB/perm/dual from `blossom_pool_duals`; LB_1 from Proposition 3 (max of the two one-sided repairs); $\text{LB}_2 = \sum_i u_i + \sum_j v_j$ from a converged Sinkhorn run at $\varepsilon = 0.05 \overline{C}$ per configuration (shared across k); $\text{LB} = \max(\text{LB}_1, \text{LB}_2)$.

B.3 Parameters

Marginal tolerance 10^{-9} on the dense grid (application hook: 10^{-7} , since an integer plan is produced regardless); time caps 400 s (dense grid), 240 s (certificate-study Sinkhorn runs); Blossom VI pool timeout 120 s (never hit); costs rounded to milliunits for the integer solvers; all timings single-threaded wall time, same machine, same process order, single run per cell.

C Reproduction

All code, data, and results are open artifacts under the MIT license, available at <https://github.com/dimkadimon/OT-Blossom>. The repository layout:

ot-study/otlib.py	# exact (LAPJV, Blossom pool, 1-D quantile),
	# sinkhorn, greenkhorn, dual repair, certificate
ot-study/run_ot_study.py	# Part A/B tables -> results/ot_study.md
ot-study/run_greenkhorn_study.py	# Greenkhorn-vs-Sinkhorn sweep
ot-study/run_pool_cert_study.py	# 15-config x 3-k certificate study
ot-study/app_hook_robots.py	# multi-robot task-allocation experiment
paper/	# LaTeX source and figs for this paper

Each script caches results (JSON) and re-renders in seconds; a full cold re-run takes ≈ 25 min (Greenkhorn grid), ≈ 20 min (certificate study), ≈ 6 min (application hook) on the testbed. The Blossom VI binary used for pool solves is available from its authors [6]; our driver uses the edge-list interface with the `-duals` flag.

D Prompts that directed this work

This appendix summarizes, in chronological order, the prompts used by D. K. to direct the work reported here. Prompts marked (verbatim) are quoted exactly; earlier prompts are paraphrased from the conversation record because their exact wording was not preserved.

1. (paraphrase) Opening question: can the Blossom (matching) algorithm be used for the TSP, and if so how—setting up the study of Edmonds’ algorithm as a component of TSP algorithms.
2. (paraphrase) Requests to implement and validate a Christofides heuristic using Blossom for the matching step against exact optima on small metric instances, and to benchmark the pre-shortcut multigraph.
3. (paraphrase) Request for a head-to-head comparison of Blossom V and Blossom VI (matching costs and wall times), including on dense complete graphs.
4. (verbatim, mid-task) “Actually you can now use Blossom VI” with a link to the Blossom VI repository—direction to make Blossom VI the default engine (retaining Blossom V for the head-to-head).
5. (paraphrase) Follow-up: “perturb the 2-matching, merge its cycles into one tour”—with the standing requirement that the merge be efficient and come close to the optimal tour, and that results be compared to state-of-the-art methods.
6. (paraphrase) Request for rigorous lower bounds via matching (2-factor / cycle-cover relaxations) to certificate the gap of the merge-based tours.

7. (verbatim) “Where else is Blossom useful?”—advisory question that produced the shortlist of exact-matching applications, of which optimal transport was ranked #1.
8. (verbatim) “do #1”—selection of the OT study as the next project.
9. (verbatim) “Yes build it. Continue clearing the publication bar”— approval to build the measured SOTA (Greenkhorn) baseline and the seed/distribution robustness sweep.
10. (verbatim) “Continue bridging the publication bar”—direction to complete the remaining two items: a pool solver with a gap certificate, and an application hook where the discrete plan is the deliverable.
11. (verbatim, this paper) “excellent! can you prepare an arXiv paper about it with references and proofs. Make it professional and publishable. Add an appendix section where you summarise the prompts I have used here. Can you add me to the author list (Dmitry Kamenetsky with email dkamenetsky@gmail.com) and the AI. Mention that all the work was done by AI with the suggestion from me. Also convert it to a pdf so I can look at it”